UNIT I

INTRODUCTION OF SOFTWARE ENGINEERING:

Software is a program or set of programs containing instructions that provide the desired functionality. **Engineering** is the process of designing and building something that serves a particular purpose and finds a cost-effective solution to problems.

What is Software Engineering?

Software Engineering is the process of designing, developing, testing, and maintaining software. It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software.

1. Software engineering includes a variety of techniques, tools, and methodologies, including requirements analysis, design, testing, and maintenance.

2. It is a rapidly evolving field, and new tools and technologies are constantly being developed to improve the software development process.

3. By following the principles of software engineering and using the appropriate tools and methodologies, software developers can create high-quality, reliable, and maintainable software that meets the needs of its users.

4. Software Engineering is mainly used for large projects based on software systems rather than single programs or applications.

5. The main goal of Software Engineering is to develop software applications for improving quality, budget, and time efficiency.

6. Software Engineering ensures that the software that has to be built should be consistent, correct, also on budget, on time, and within the required requirements.

Key Principles of Software Engineering

1. **Modularity**: Breaking the software into smaller, reusable components that can be developed and tested independently.

2. **Abstraction**: Hiding the implementation details of a component and exposing only the necessary functionality to other parts of the software.

3. **Encapsulation**: Wrapping up the data and functions of an object into a single unit, and protecting the internal state of an object from external modifications.

4. **Reusability**: Creating components that can be used in multiple projects, which can save time and resources.

5. **Maintenance**: Regularly updating and improving the software to fix bugs, add new features, and address security vulnerabilities.

6. **Testing**: Verifying that the software meets its requirements and is free of bugs.

7. **Design Patterns**: Solving recurring problems in software design by providing templates for solving them.

8. **Agile methodologies:** Using iterative and incremental development processes that focus on customer satisfaction, rapid delivery, and flexibility.

9. **Continuous Integration & Deployment:** Continuously integrating the code changes and deploying them into the production environment.

Main Attributes of Software Engineering

Software Engineering is a systematic, disciplined, quantifiable study and approach to the design, development, operation, and maintenance of a software system. There are four main Attributes of Software Engineering.

1. **Efficiency:** It provides a measure of the resource requirement of a software product efficiently.

2. **Reliability:** It assures that the product will deliver the same results when used in similar working environment.

3. **Reusability:** This attribute makes sure that the module can be used in multiple applications.

4. **Maintainability:** It is the ability of the software to be modified, repaired, or enhanced easily with changing requirements.

What Careers Are There in Software Engineering?

A degree in software engineering and relevant experience can be utilized to explore several computing job choices. Software engineers have the opportunity to seek well -paying careers and professional progress, although their exact possibilities may vary depending on their particular school, industry, and region.

Following are the job choices in software engineering:

- SWE (Software Engineer)

- SDE (Software Development Engineer)

- Web Developer

- Quality Assurance Engineer

- Web Designer
- Software Test Engineer
- Cloud Engineer ·
- Front-End Developer
- Back-End Developer
- DevOps Engineer.
- Security Engineer.

1.1 The Evolving Role of Software:

Dual Role of Software

There is a dual role of software in the industry. The first one is as a product and the other one is as a vehicle for delivering the product. We will discuss both of them.

1. As a Product

- It delivers computing potential across networks of Hardware.
- It enables the Hardware to deliver the expected functionality.
- It acts as an information transformer because it produces, manages, acquires, modifies, displays, or transmits information.

2. As a Vehicle for Delivering a Product

- It provides system functionality (e.g., payroll system).
- It controls other software (e.g., an operating system).
- It helps build other software (e.g., software tools).

Objectives of Software Engineering

1. **Maintainability:** It should be feasible for the software to evolve to meet changing requirements.
2. **Efficiency:** The software should not make wasteful use of computing devices such as memory, processor cycles, etc.

3. **Correctness:** A software product is correct if the different requirements specified in the SRS Document have been correctly implemented.

4. **Reusability:** A software product has good reusability if the different modules of the product can easily be reused to develop new products.

5. **Testability:** Here software facilitates both the establishment of test criteria and the evaluation of the software concerning those criteria.

6. **Reliability:** It is an attribute of software quality. The extent to which a program can be expected to perform its desired function, over an arbitrary time period.

7. **Portability:** In this case, the software can be transferred from one computer system or environment to another.

8. **Adaptability:** In this case, the software allows differing system constraints and the user needs to be satisfied by making changes to the software.

9. **Interoperability:** Capability of 2 or more functional units to process data cooperatively.

Advantages of Software Engineering

There are several advantages to using a systematic and disciplined approach to software development, such as:

1. **Improved Quality:** By following established software engineering principles and techniques, the software can be developed with fewer bugs and higher reliability.

2. **Increased Productivity:** Using modern tools and methodologies can streamline the development process, allowing developers to be more productive and complete projects faster.

3. **Better Maintainability:** Software that is designed and developed using sound software engineering practices is easier to maintain and update over time.

4. **Reduced Costs:** By identifying and addressing potential problems early in the development process, software engineering can help to reduce the cost of fixing bugs and adding new features later on.

5. **Increased Customer Satisfaction:** By involving customers in the development process and developing software that meets their needs, software engineering can help to increase customer satisfaction.

6. **Better Team Collaboration:** By using Agile methodologies and continuous integration, software engineering allows for better collaboration among development teams.

7. **Better Scalability**: By designing software with scalability in mind, software engineering can help to ensure that software can handle an increasing number of users and transactions.

8. **Better Security:** By following the Software Development Life Cycle (SDLC) and performing security testing, software engineering can help to prevent security breaches and protect sensitive data.

In summary, software engineering offers a structured and efficient approach to software development, which can lead to higher-quality software that is easier to maintain and adapt to changing requirements. This can help to improve customer satisfaction and reduce costs, while also promoting better collaboration among development teams.

Disadvantages of Software Engineering

While Software Engineering offers many advantages, there are also some potential disadvantages to consider:

1. **High upfront costs:** Implementing a systematic and disciplined approach to software development can be resource-intensive and require a significant investment in tools and training.

2. **Limited flexibility:** Following established software engineering principles and methodologies can be rigid and may limit the ability to quickly adapt to changing requirements.

3. **Bureaucratic**: Software Engineering can create an environment that is bureaucratic, with a lot of processes and paperwork, which may slow down the development process.

4. **Complexity**: With the increase in the number of tools and methodologies, software engineering can be complex and difficult to navigate.

5. **Limited creativity:** The focus on structure and process can stifle creativity and innovation among developers.

6. **High learning curve:** The development process can be complex, and it requires a lot of learning and training, which can be challenging for new developers.

7. **High dependence on tools:** Software engineering heavily depends on the tools, and if the tools are not properly configured or are not compatible with the software, it can cause issues.

8. **High maintenance**: The software engineering process requires regular maintenance to ensure that the software is running efficiently, which can be costly and time-consuming.

1.2 Characteristics of software engineering:

Software engineering is a discipline that applies engineering principles to the development of software systems. It focuses on designing, developing, testing, maintaining, and evolving software products while ensuring that they are reliable, efficient, and meet user requirements. The characteristics of software engineering help define how this process works and contribute to the success of a software project.

Here are the key characteristics of software engineering:

1. Systematic Approach

- Software engineering follows a well-defined, structured approach to developing software, often through methodologies such as **Waterfall**, **Agile**, **DevOps**, and **Scrum**. This ensures that each phase of development (requirements gathering, design, implementation, testing, maintenance) is carefully planned and executed.

2. Focus on Quality

- Software engineering prioritizes quality at every stage of development. This includes writing high-quality code, performing rigorous testing, and adhering to standards and best practices.
- Key quality attributes include:
    - **Reliability:** Software should perform its intended function under expected conditions without failure.
    - **Efficiency:** Software should make optimal use of resources (CPU, memory, network bandwidth).
    - **Usability:** Software should be user-friendly and intuitive.
    - **Maintainability:** The software should be easy to update, extend, or fix over time.

3. Scalability

- Software systems should be designed to **scale** as they grow in size or complexity. This includes the ability to handle increasing numbers of users, larger datasets, or more complex processes without degrading performance.

4. Modularity

- Software should be **modular**, meaning that it is divided into smaller, manageable components (modules) that can be developed, tested, and maintained independently. This helps in reducing complexity and improving code reusability and maintainability.

5. Reusability

- A fundamental characteristic of software engineering is the emphasis on

reusability. Engineers aim to create reusable components and libraries, reducing the need to write redundant code and speeding up the development process.

6. Documentation

- Software engineering involves maintaining thorough **documentation** for every stage of the software development lifecycle (SDLC). This includes:
  - **Requirements documentation**
  - **Design specifications**
  - **Test plans and results**
  - **User manuals**
- Documentation ensures that future developers can understand, modify, or extend the software and that stakeholders can track progress.

7. Testing and Validation

- Comprehensive **testing** is a core component of software engineering. It involves verifying that the software functions as expected and meets the user requirements. Common types of testing include:
  - **Unit testing** (testing individual components)
  - **Integration testing** (testing interaction between components)
  - **System testing** (testing the entire system)
  - **Acceptance testing** (verifying software meets user needs)
- **Validation** ensures the product meets business objectives and user expectations.

8. User-Centered Design

- Software engineering emphasizes **user-centered design**, meaning the software is developed with a focus on meeting the needs and expectations of the end-users. This often involves gathering and analyzing user requirements and incorporating feedback throughout the development process.

9. Risk Management

- In software engineering, **risk management** involves identifying, assessing, and mitigating risks during the development process. These could be technical risks (e.g., software failure), organizational risks (e.g., resource allocation), or market risks (e.g., changing customer requirements).
- Methods like **prototyping**, **incremental development**, and **continuous integration** can help manage and reduce these risks.

10. Collaboration and Teamwork

- **Collaboration** is critical in software engineering, as developing complex systems often requires teams of engineers, designers, testers, and business stakeholders working together. Agile practices like **Scrum** and **Kanban** promote teamwork and regular communication among team members to

ensure alignment and timely delivery.

11. Iteration and Incremental Development

- Software development is often done in **iterations** (in Agile frameworks) or through **incremental development**, where the software is built in small, manageable parts. Each increment adds value and functionality, allowing for user feedback and improvement at each stage.

12. Flexibility and Adaptability

- The ability to adapt to changing requirements, especially in the face of new technologies or evolving user needs, is another key characteristic of software engineering. Agile methodologies emphasize flexibility, allowing engineers to adjust their approach as new information emerges.

13. Automation

- Software engineering uses **automation** tools to streamline repetitive tasks, such as code deployment, testing, and integration. Tools like **CI/CD** (Continuous Integration/Continuous Deployment) help automate the process of building, testing, and releasing software, improving productivity and reducing human errors.

14. Ethical Considerations

- Ethical considerations are central to software engineering, particularly in areas like **data privacy**, **security**, and **intellectual property**. Engineers must consider the social and ethical implications of their work and ensure that the software complies with legal and ethical standards.
- Ensuring the security of software applications against hacking, data breaches, and other threats is paramount.

15. Cost-Effectiveness

- Software engineering emphasizes developing software in a **cost-effective** manner. This includes using resources wisely, choosing appropriate technologies, and optimizing code to ensure that the software delivers value for its cost.
- **Cost-benefit analysis** is often performed to ensure the value of the software justifies the development and operational costs.

16. Maintenance and Evolution

- Once software is deployed, it requires ongoing **maintenance** to fix bugs, update features, and adapt to changes in technology or user needs. Software engineering places a strong emphasis on maintaining and evolving software throughout its lifecycle.
- **Legacy systems** and their integration with new technologies are common

challenges.

17. Interdisciplinary Knowledge

- Software engineering often requires knowledge from other fields, such as business (to understand user requirements), mathematics (for algorithms and data structures), and design (for user interfaces and experience). A successful software engineer should be proficient not only in coding but also in understanding and applying principles from these disciplines.

18. Continuous Learning and Innovation

- Software engineering is a field of **continuous learning**, with new tools, languages, frameworks, and methodologies emerging regularly. Engineers must stay updated with the latest developments in technology and practices to remain competitive and innovative.

19. Performance and Optimization

- Software engineers focus on optimizing the performance of the software. This includes making sure the software runs efficiently in terms of speed and resource consumption, especially when dealing with large datasets, heavy traffic, or real-time systems.
- Techniques like **profiling**, **load testing**, and **code optimization** are used to ensure high performance.

1.3 APPLICATIONS OF SOFTWARE ENGINEERING:

Software engineering has a wide range of applications across various industries and fields. As technology continues to evolve, software engineering plays a critical role in shaping the digital world we interact with daily. Below are some of the major applications of software engineering:

1. Web Development

- **Frontend Development:** Involves creating user interfaces for websites or web applications, using technologies like HTML, CSS, JavaScript, React, and Angular.
- **Backend Development:** Deals with server-side logic, databases, and APIs. Technologies include Node.js, Ruby on Rails, Python (Django, Flask), Java (Spring), and more.
- **Full-Stack Development:** A combination of frontend and backend development.

2. Mobile Application Development

- **Native Mobile Apps:** Developing apps specifically for Android (Java/Kotlin) or iOS (Swift/Objective-C).
- **Cross-Platform Apps:** Using frameworks like Flutter, React Native, or Xamarin to create apps that run on both Android and iOS with a single codebase.
- **Mobile Games:** Software engineers also work on developing games for mobile platforms, using game engines like Unity or Unreal Engine.

3. Cloud Computing

- **Cloud Infrastructure and Services:** Software engineers design and maintain cloud-based systems (Amazon Web Services, Microsoft Azure, Google Cloud) to provide scalable solutions for data storage, computing power, and application deployment.
- **Cloud-Native Development:** Creating applications that are built specifically for cloud environments, utilizing microservices, containers (Docker), and orchestration tools like Kubernetes.

4. Enterprise Software

- **Enterprise Resource Planning (ERP):** Developing software to integrate core business processes, such as SAP, Oracle, and Microsoft Dynamics.
- **Customer Relationship Management (CRM):** Software systems like Salesforce or HubSpot that help businesses manage customer interactions.
- **Business Intelligence (BI):** Creating tools for data analysis, reporting, and decision-making, such as Tableau, Power BI, or custom solutions.

5. Embedded Systems

- Software engineers design software for devices with embedded processors, often found in industries like automotive, healthcare, consumer electronics, and industrial control systems. This includes:
    - **Internet of Things (IoT):** Smart home devices, wearable technology, smart appliances.
    - **Automotive Systems:** Software for self-driving cars, in-car navigation, or advanced driver-assistance systems (ADAS).
    - **Medical Devices:** Software for pacemakers, infusion pumps, or diagnostic equipment.

6. Artificial Intelligence (AI) and Machine Learning (ML)

- **AI and ML Algorithms:** Software engineers develop systems that can learn from data, such as recommendation engines, natural language processing (NLP) systems (chatbots, language translators), computer vision, and autonomous systems.
- **Data Science Tools:** Building frameworks and tools for processing large datasets, such as TensorFlow, PyTorch, and Scikit-learn.

7. Cybersecurity

- **Security Software Development:** Designing software tools that help protect systems, networks, and data from cyberattacks, such as firewalls, antivirus software, and encryption tools.
- **Vulnerability Assessment and Penetration Testing:** Engineers develop systems and tools for identifying weaknesses in networks and systems, as well as tools for ethical hacking.

8. Gaming and Entertainment

- **Video Games:** Game development is a major area for software engineers, particularly using game engines like Unity or Unreal Engine. It involves gameplay mechanics, graphics, AI, and multiplayer systems.
- **Virtual Reality (VR) and Augmented Reality (AR):** Engineers create immersive experiences for gaming, education, or industrial applications using VR/AR technologies.

9. Data Analytics and Big Data

- **Data Processing and Analytics:** Software engineers develop systems that handle and process massive datasets using tools like Hadoop, Spark, and data lakes.
- **Business Intelligence and Reporting Tools:** Creating applications that aggregate and analyze data to help businesses make data-driven decisions.

10. DevOps and Automation

- **Continuous Integration/Continuous Deployment (CI/CD):** Automating the process of integrating code changes and deploying software applications, ensuring faster delivery of features and bug fixes.
- **Infrastructure as Code (IaC):** Using tools like Terraform or Ansible to automate the provisioning of infrastructure.

11. Blockchain and Cryptography

- **Blockchain Development:** Software engineers design decentralized applications (DApps) and smart contracts, primarily using technologies like Ethereum or Solana.
- **Cryptography:** Building systems that ensure secure communication, data protection, and digital currency solutions.

12. Artificial Reality & Virtual Reality (AR/VR)

- Software engineering in AR/VR enables new applications in education, training, healthcare, real estate, and gaming. This includes creating immersive 3D environments, simulations, and interaction systems for users.

13. Healthcare IT

- **Health Information Systems (HIS):** Developing software for managing patient

data, hospital operations, or electronic medical records (EMR).
- **Telemedicine:** Engineers create platforms that enable remote consultations between patients and healthcare providers.
- **Health Monitoring Devices:** Software for wearable health devices like fitness trackers, smartwatches, and medical sensors.

14. Autonomous Systems

- **Self-Driving Cars:** Software engineers develop the algorithms for autonomous vehicles, including navigation, decision-making, and safety protocols.
- **Drones and Robotics:** Engineering software that controls and monitors autonomous drones, robots, or other devices for various applications, from delivery to search-and-rescue operations.

15. Natural Language Processing (NLP)

- **Chatbots and Virtual Assistants:** Software engineers develop conversational AI systems, like Siri, Alexa, or customer support bots.
- **Text Analysis and Translation:** Engineers build systems that can understand, translate, and process human languages in a variety of applications.

16. Financial Software

- **FinTech Applications:** Software engineers develop applications for financial management, such as online banking, stock trading platforms, and budgeting tools.
- **Cryptocurrency Platforms:** Software for trading, securing, and managing digital assets like Bitcoin, Ethereum, or stablecoins.

17. Social Media and Networking

- Software engineers design systems that power social media platforms, handle user interactions, store massive amounts of data, and recommend content based on algorithms.

18. Educational Software

- **E-learning Platforms:** Development of online learning tools, course management systems (LMS), and educational games.
- **Simulation Software:** Tools for training professionals in fields like aviation, healthcare, and engineering.

19. Artificial Intelligence in Robotics

- **AI-Powered Robots:** Engineers create robots that use AI for tasks like manufacturing automation, warehouse management, or surgical assistance.

20. Distributed Systems and Networking

- **Network Protocols and Communication Systems:** Software engineers develop communication protocols, peer-to-peer networking, and systems that manage the flow of data across networks.

21. Automation and RPA (Robotic Process Automation)

- **Automation Tools:** Software engineers develop systems for automating repetitive tasks in industries like finance, healthcare, and human resources using tools like UiPath or Blue Prism.

1.4 SOFTWARE EVOLUTION:

Software Evolution is a term that refers to the process of developing software initially, and then timely updating it for various reasons, i.e., to add new features or to remove obsolete functionalities, etc. This article focuses on discussing Software Evolution in detail.

What is Software Evolution?

The software evolution process includes fundamental activities of change analysis, release planning, system implementation, and releasing a system to customers.

1. The cost and impact of these changes are accessed to see how much the system is affected by the change and how much it might cost to implement the change.

2. If the proposed changes are accepted, a new release of the software system is planned.

3. During release planning, all the proposed changes (fault repair, adaptation, and new functionality) are considered.

4. A design is then made on which changes to implement in the next version of the system.

5. The process of change implementation is an iteration of the development process where the revisions to the system are designed, implemented, and tested.

Necessity of Software Evolution

Software evaluation is necessary just because of the following reasons:

1. **Change in requirement with time:** With time, the organization's needs and modus Operandi of working could substantially be changed so in this frequently changing time the tools(software) that they are using need to

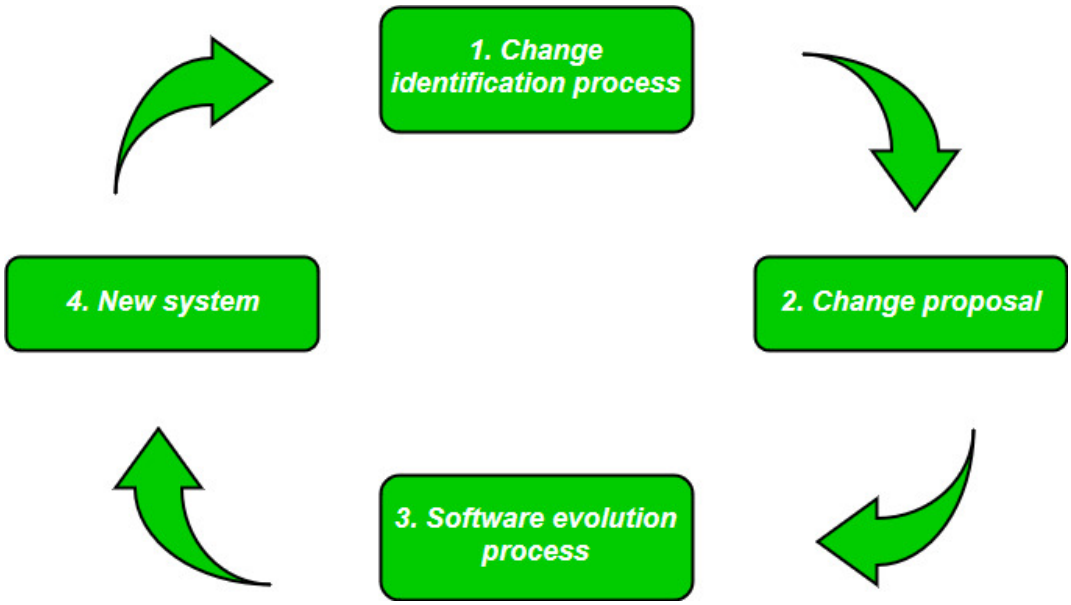
change to maximize the performance.

2. **Environment change:** As the working environment changes the things(tools) that enable us to work in that environment also changes proportionally same happens in the software world as the working environment changes then, the organizations require reintroduction of old software with updated features and functionality to adapt the new environment.

3. **Errors and bugs:** As the age of the deployed software within an organization increases their preciseness or impeccability decrease and the efficiency to bear the increasing complexity workload also continually degrades. So, in that case, it becomes necessary to avoid use of obsolete and aged software. All such obsolete Pieces of software need to undergo the evolution process in order to become robust as per the workload complexity of the current environment.

4. **Security risks:** Using outdated software within an organization may lead you to at the verge of various software-based cyberattacks and could expose your confidential data illegally associated with the software that is in use. So, it becomes necessary to avoid such security breaches through regular assessment of the security patches/modules are used within the software. If the software isn't robust enough to bear the current occurring Cyber attacks so it must be changed (updated).

5. **For having new functionality and features:** In order to increase the performance and fast data processing and other functionalities, an organization need to continuously evolute the software throughout its life cycle so that stakeholders & clients of the product could work efficiently.



Software Evolution

Laws used for Software Evolution

1. Law of Continuing Change

This law states that any software system that represents some real-world reality undergoes continuous change or become progressively less useful in that environment.

2. Law of Increasing Complexity

As an evolving program changes, its structure becomes more complex unless effective efforts are made to avoid this phenomenon.

3. Law of Conservation of Organization Stability

Over the lifetime of a program, the rate of development of that program is approximately constant and independent of the resource devoted to system development.

4. Law of Conservation of Familiarity

This law states that during the active lifetime of the program, changes made in the successive release are almost constant.

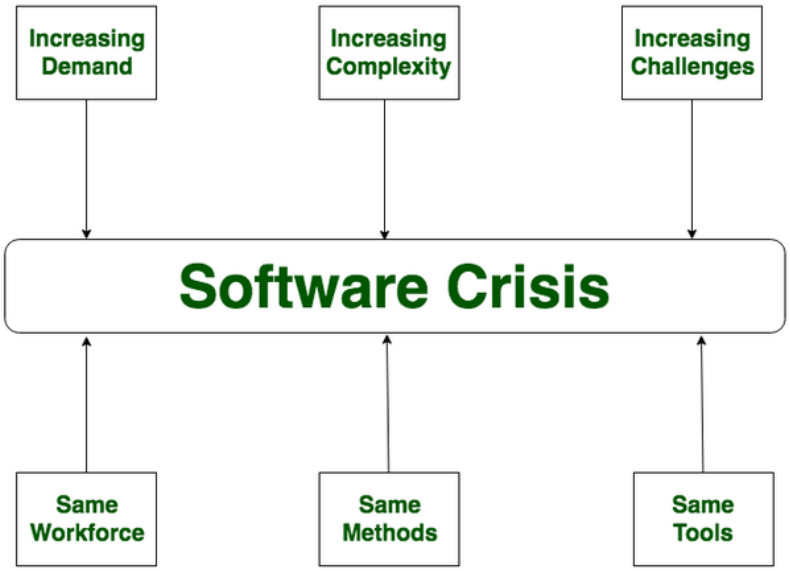
1.5 SOFTWARE CRISES AND HORIZON:

The term “software crisis” refers to the numerous challenges and difficulties faced by the software industry during the 1960s and 1970s. It became clear that old methods of developing software couldn’t keep up with the growing complexity and demands of new projects. This led to high costs, delays, and poor-quality software. New methodologies and tools were needed to address these issues.

What is Software Crisis?

Software Crisis is a term used in computer science for the difficulty of writing useful and efficient computer programs in the required time. The software crisis was due to using the same workforce, same methods, and same tools even though rapidly increasing software demand, the complexity of software, and software challenges. With the increase in software complexity, many software problems arose because existing methods were insufficient.

Suppose we use the same workforce, same methods, and same tools after the fast increase in software demand, software complexity, and software challenges. In that case, there arise some issues like software budget problems, software efficiency problems, software quality problems, software management, and delivery problems, etc. This condition is called a **Software Crisis.**



Software Crisis

Causes of Software Crisis

Following are the causes of Software Crisis:

- The cost of owning and maintaining software was as expensive as developing the software.
- At that time Projects were running overtime.
- At that time Software was very inefficient.
- The quality of the software was low quality.
- Software often did not meet user requirements.
- The average software project overshoots its schedule by half.
- At that time Software was never delivered.
- Non-optimal resource utilization.
- Challenging to alter, debug, and enhance.
- The software complexity is harder to change.

Factors Contributing to Software Crisis

Factor Contributing to Software Crisis are:

- Poor project management.
- Lack of adequate training in software engineering.
- Less skilled project members.
- Low productivity improvements.

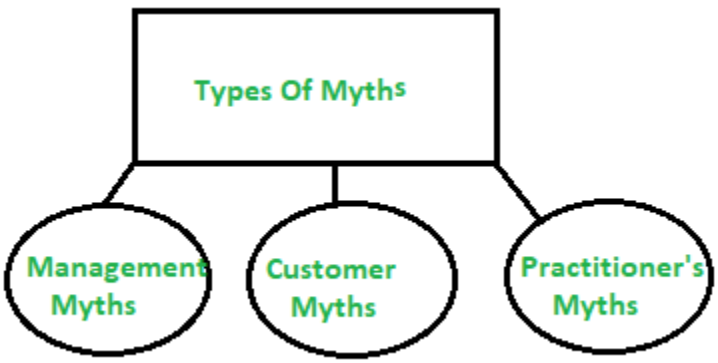
Solution of Software Crisis:

There is no single solution to the crisis. One possible solution to a software crisis is *Software Engineering* because software engineering is a systematic, disciplined, and quantifiable approach. For preventing software crises, there are some guidelines:

- Reduction in software over budget.
- The quality of the software must be high.
- Less time is needed for a software project.
- Experienced and skilled people working on the software project.
- Software must be delivered.
- Software must meet user requirements.

1.6 Software Myths:

Most, experienced experts have seen myths or superstitions (false beliefs or interpretations) or misleading attitudes (naked users) which creates major problems for management and technical people. The types of software-related myths are listed below.



`Types of Software Myths

(i) Management Myths:

Myth 1:

We have all the standards and procedures available for software development.

Fact:

- Software experts do not know all the requirements for the software development.
- And all existing processes are incomplete as new software development is based on new and different problem.

Myth 2:

The addition of the latest hardware programs will improve the software development.

Fact:

- The role of the latest hardware is not very high on standard software development; instead (CASE) Engineering tools help the computer, they are more important than hardware to produce quality and productivity.
- Hence, the hardware resources are misused.

Myth 3:

- With the addition of more people and program planners to Software development can help meet project deadlines (If lagging behind).

Fact:

- If software is late, adding more people will merely make the problem worse. This is because the people already working on the project now need to spend time educating the newcomers, and are thus taken away from their work. The newcomers are also far less productive than the existing software engineers, and so the work put into training them to work on the software does not immediately meet with an appropriate reduction in work.

(ii)Customer Myths:

The customer can be the direct users of the software, the technical team, marketing / sales department, or other company. Customer has myths leading to false expectations (customer) & that's why you create dissatisfaction with the developer.

Myth 1:

A general statement of intent is enough to start writing plans (software development) and details of objectives can be done over time.

Fact:

- Official and detailed description of the database function, ethical performance,

communication, structural issues and the verification process are important.

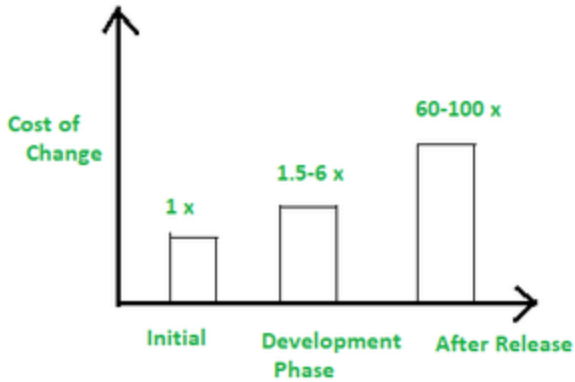
- Unambiguous requirements (usually derived iteratively) are developed only through effective and continuous communication between customer and developer.

Myth 2:

Software requirements continually change, but change can be easily accommodated because software is flexible

Fact:

- It is true that software requirements change, but the impact of change varies with the time at which it is introduced. When requirements changes are requested early (before design or code has been started), the cost impact is relatively small. However, as time passes, the cost impact grows rapidly—resources have been committed, a design framework has been established, and change can cause upheaval that requires additional resources and major design modification.



Cost of Change

Different Stages of Myths

(iii)Practitioner's Myths:

Myths 1:

They believe that their work has been completed with the writing of the plan.

Fact:

- It is true that every 60-80% effort goes into the maintenance phase (as of the latter software release). Efforts are required, where the product is available first delivered to customers.

Myths 2:

There is no other way to achieve system quality, until it is “running”.

Fact:

- Systematic review of project technology is the quality of effective software verification method. These updates are quality filters and more accessible than test.

Myth 3:

An operating system is the only product that can be successfully exported project.

Fact:

- A working system is not enough, the right document brochures and booklets are also required to provide guidance & software support.

Myth 4:

Engineering software will enable us to build powerful and unnecessary document & always delay us.

Fact:

- Software engineering is not about creating documents. It is about creating a quality product. Better quality leads to reduced rework. And reduced rework results in faster delivery times